```
"""
ARKHEIA-CPS — Contracts Router
POST /contracts/auto     → Full FIA →
OIA-PEM → RTCSA pipeline for auto
POST /contracts/housing  → Full FIA →
OIA-PEM → RTCSA pipeline for housing
"""
from fastapi import APIRouter, Depends,
HTTPException, status
from sqlalchemy.orm import Session
from datetime import datetime

from app.db import get_db
from app.schemas.auto import
AutoContractIn, AutoAnalysisResponse
from app.schemas.housing import
HousingContractIn, HousingAnalysisResponse
from app.schemas.common import
VerticalType, RiskLevel, Alert,
PEMEvaluation
from app.services.fia_auto import
FIAAutoService
from app.services.fia_housing import
FIAHousingService
from app.services.rt_csa_auto import
RTCSAAutoService
from app.services.rt_csa_housing import
RTCSAHousingService

router = APIRouter(prefix="/contracts",
```

```
tags=["Contracts"])


# ── POST /contracts/auto
──────────────────────────────────────────────
──

@router.post("/auto",
response_model=AutoAnalysisResponse,
status_code=status.HTTP_201_CREATED)
def submit_auto_contract(payload:
AutoContractIn, db: Session =
Depends(get_db)):
    """
    Submit an auto contract for full FIA →
OIA-PEM → RTCSA analysis.

    Pipeline:
    1. FIA: Structure all entities
(consumer, dealer/lender, vehicle, terms)
    2. OIA-PEM: Evaluate affordability,
APR, fees, add-ons, total cost
    3. Perfection Law: Analyze lien timing
and sequencing
    4. RTCSA: Score risk dimensions,
generate alerts and explanations
    5. Persist AnalysisResult to database
    6. Return full analysis response
    """
    try:
        # Layer 1: FIA
        fia            = FIAAutoService(db)
        contract, auto =
fia.structure(payload)
```

```
        # Layer 3: RTCSA (orchestrates OIA-
PEM + Perfection Law internally)
        rtcsa  = RTCSAAutoService(db)
        result = rtcsa.analyze(contract,
auto, payload)

        db.commit()

        return AutoAnalysisResponse(
            contract_id=contract.id,
            vertical=VerticalType.AUTO,

overall_risk_score=result.overall_risk_scor
e,

risk_level=RiskLevel(result.risk_level),

dimension_scores=result.dimension_scores,
            triggered_alerts=[
                Alert(
                    code=a.code,
                    severity=a.severity,
                    message=a.message,
                    statute=a.statute,
                )
                for a in result.alerts
            ],

explanations=result.explanations,
            pem_evaluations=[
                PEMEvaluation(

pattern_family=e.pattern_family,
```

```
pattern_key=e.pattern_key,

expected_min=e.expected_min,

expected_max=e.expected_max,

actual_value=e.actual_value,
                    status=e.status,

explanation=e.explanation,
                )
                for e in
result.pem_evaluations
            ],
            statutes=result.statutes,
            analyzed_at=datetime.utcnow(),

perfection_days=result.perfection_days,
            perfection_flags=[e.pattern_key
for e in result.pem_evaluations
                                if
e.pattern_family == "PERFECTION" and
e.status == "FAIL"],

illegal_patterns=result.illegal_patterns,
        )

    except Exception as e:
        db.rollback()
        raise HTTPException(

status_code=status.HTTP_500_INTERNAL_SERVER
_ERROR,
```

```
            detail=f"Auto contract analysis
failed: {str(e)}"
        )


# ── POST /contracts/housing
──────────────────────────────────────────


@router.post("/housing",
response_model=HousingAnalysisResponse,
status_code=status.HTTP_201_CREATED)
def submit_housing_contract(payload:
HousingContractIn, db: Session =
Depends(get_db)):
    """
    Submit a housing/lease contract for
full FIA → OIA-PEM → RTCSA analysis.

    Pipeline:
    1. FIA: Structure all entities (tenant,
landlord, property, lease terms)
    2. OIA-PEM: Evaluate rent-to-income,
fees, clauses, eviction, habitability
    3. RTCSA: Score risk dimensions,
generate alerts and explanations
    4. Persist AnalysisResult to database
    5. Return full analysis response
    """
    try:
        # Layer 1: FIA
        fia                 =
FIAHousingService(db)
        contract, housing    =
fia.structure(payload)
```

```
        # Layer 3: RTCSA
        rtcsa  = RTCSAHousingService(db)
        result = rtcsa.analyze(contract,
housing, payload)

        db.commit()

        return HousingAnalysisResponse(
            contract_id=contract.id,
            vertical=VerticalType.HOUSING,

overall_risk_score=result.overall_risk_scor
e,

risk_level=RiskLevel(result.risk_level),

dimension_scores=result.dimension_scores,
            triggered_alerts=[
                Alert(
                    code=a.code,
                    severity=a.severity,
                    message=a.message,
                )
                for a in result.alerts
            ],

explanations=result.explanations,
            pem_evaluations=[
                PEMEvaluation(

pattern_family=e.pattern_family,

pattern_key=e.pattern_key,
```

```
expected_min=e.expected_min,

expected_max=e.expected_max,

actual_value=e.actual_value,
                    status=e.status,

explanation=e.explanation,
                )
                for e in
result.pem_evaluations
            ],
            statutes=result.statutes,
            analyzed_at=datetime.utcnow(),

total_movein_cost=result.total_movein_cost,

total_monthly_recurring=result.total_monthl
y_recurring,

illegal_clauses_found=result.illegal_clause
s,

predatory_flags=result.predatory_flags,
        )

    except Exception as e:
        db.rollback()
        raise HTTPException(

status_code=status.HTTP_500_INTERNAL_SERVER
_ERROR,
            detail=f"Housing contract
```

```
analysis failed: {str(e)}"
        )
```